清華大學
Tsinghua University

2026秋季

计算机系统概论

Introduction to Computer Systems

浮 点 数









Zhang，Youhui（张悠慧）

zyh02@tsinghua.edu.cn https://craft.cs.tsinghua.edu.cn/zh/#lab

计算机体系结构

```c
int array[4] = {1, 2, 3, 4};
static int brray[4] = {1, 2, 3, 4};

static int intra_sum (int x[4], int y)
{
   return x[y-1];
}
int main()
{
    int val = intra_sum(array, 3) + inter_sum(brray, 3);
    return val;
}
```

main.c

编译

```
intra_sum:
        movslq  %esi, %rsi
        movl    -4(%rdi,%rsi,4), %eax
        ret
main:
        pushq   %rbx
        movl    $3, %esi
        movl    $array, %edi
        call    intra_sum
        movl    %eax, %ebx// val -> ebx
        movl    $3, %esi
        movl    $brray, %edi
        movl    $0, %eax
        call    inter_sum
        addl    %ebx, %eax// val -> eax
        popq    %rbx
        ret

brray:
        .long   1
        .long   2
        .long   3
        .long   4
array:
        .long   1
        .long   2
        .long   3
        .long   4
```

汇编指令

main.o

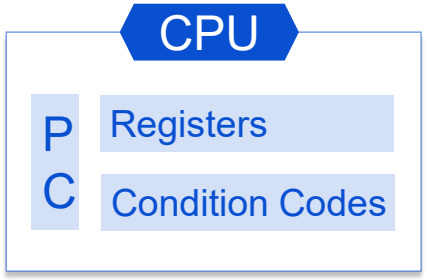
链接

```
00000000004004de <main>:
  4004de:       53                      push   %rbx
  4004df:       be 03 00 00 00          mov    $0x3,%esi
  4004e4:       bf 40 10 60 00          mov    $0x601040,%edi
  4004e9:       e8 e8 ff ff ff          callq  4004d6 <intra_sum>
  4004ee:       89 c3                   mov    %eax,%ebx
  4004f0:       be 03 00 00 00          mov    $0x3,%esi
  4004f5:       bf 30 10 60 00          mov    $0x601030,%edi
  4004fa:       b8 00 00 00 00          mov    $0x0,%eax
  4004ff:       e8 04 00 00 00          callq  400508 <inter_sum>
  400504:       01 d8                   add    %ebx,%eax
  400506:       5b                      pop    %rbx
  400507:       c3                      retq
```

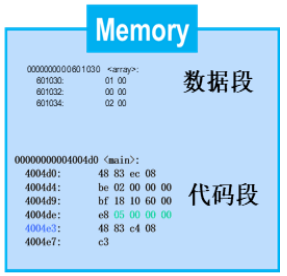
内存地址

仅给出部分内容

运行







C程序在硬件层面的表示

数据

- **整数 / 浮点数（第二讲）**
- **数组 / 结构（第五讲）**

代码

- ……

程序/数据在机器层面的表示与运行



目录

CONTENTS

01 IEEE的浮点数标准

02 Rounding（舍入）

03 C语言中的浮点数

一个C代码示例——引子

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    float position = 0.0f;
    float velocity = 0.1f;

    for (int i = 0; i < 1000; i++) {
        position += velocity;
    }

    printf("position = %.15f\n", position);
    printf("position == 100.0f ? %s\n", position == 100.0f ? "true" : "false");
    printf("error = %.15f\n", position - 100.0f);

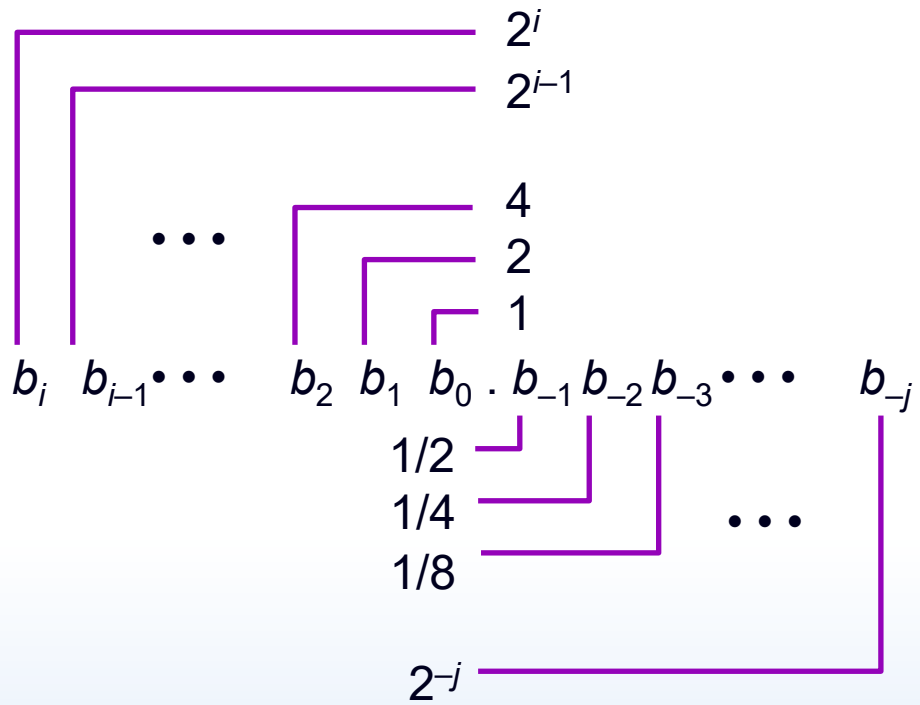
    return 0;
}
```

```
zhang@ubuntu:~/work$ gcc -O2 floating-yinzi.c
zhang@ubuntu:~/work$ ./a.out
position = 99.999046325683594
position == 100.0f ? false
error = -0.000953674316406
```

IEEE Floating Point

IEEE的754标准

1985年建立



二进制表示方式

$$\sum_{k=-j}^{i} b_k \cdot 2^k$$

浮点数示例

值	二进制表示
5.3/4	101.11_2
2.7/8	10.111_2
0.63/64	0.111111_2

局限性 只能精确地表示X/2^y这类形式的数据（回想下一开始的浮点代码示例）

值	二进制表示
1/3	$0.0101010101[01]..._2$
1/5	$0.001100110011[0011]..._2$
1/10	$0.0001100110011[0011]..._2$

一个C代码示例——引子

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    float position = 0.0f;
    float velocity = 0.1f;

    for (int i = 0; i < 1000; i++) {
        position += velocity;
        printf("position = %.15f\n", position);
    }

    printf("position = %.15f\n", position);
    printf("position == 100.0f ? %s\n", position == 100.0f ? "true" : "false");
    printf("error = %.15f\n", position - 100.0f);

    return 0;
}
```

```
gcc -O2 floating-yinzi.c
zhang@ubuntu:~/work$ ./a.out
position = 0.100000001490116
position = 0.200000002980232
position = 0.300000011920929
position = 0.400000005960464
position = 0.500000000000000
position = 0.600000023841858
position = 0.700000047683716
position = 0.800000071525574
position = 0.900000095367432
position = 1.000000119209290
position = 1.100000143051147
position = 1.200000166893005
position = 1.300000190734863
position = 1.400000214576721
position = 1.500000238418579
position = 1.600000262260437
position = 1.700000286102295
position = 1.800000309944153
position = 1.900000333786011
position = 2.000000238418579
position = 2.100000143051147
position = 2.200000047683716
```

计算机中的浮点数二进制表示

数字形式

$$(-1)^s\ M\ 2^E$$

- **符号**：s
- **有效数**：M，是一个位于区间[1.0,2.0)内的小数
- **指数（也称阶码）**：E

数字形式



- **exp域(指数域)：*E***
- **frac域(小数域，也称尾数，mantissa)：*M***

编码



单精度浮点数	指数域宽度为8比特，小数域宽度为23比特	**总共32比特**
双精度浮点数	指数域宽度为11比特，小数域宽度为52比特	**总共64比特**
扩展精度浮点数	指数域宽度为15比特，小数域宽度为63比特	**总共80比特*（1比特无用）***
半精度浮点数	指数域宽度为5比特，小数域宽度为10比特	**总共16比特**

BF16：用8bit 表示指数，7bit 表示小数（深度学习领域常用）

浮点数的类型

01

规格化浮点数

02

非规格化浮点数

03

一些特殊值

规格化浮点数 （ Normalized ）

满足条件

exp ≠ 000…0 且 exp ≠ 111…1

真实的阶码值E（即指数）需要减去一个偏置量

E = Exp – Bias

- **Exp**：指数域所表示的无符号数值

  单精度数： 127 (Exp: 1…254, E: -126…127)

  双精度数： 1023 (Exp: 1…2046, E: -1022…1023)

- Bias的取值

  Bias = $2^{e-1} - 1$，e表示exp域的位数

小数域的第一位隐含为1

M = 1.xxx…x_2

- 因此，第一位的“1”可以省去
- 当frac为000…0，值为最小，即M = 1.0
- 当frac为111…1，值为最大，即M = 2.0 – ε

ε的值?

规格化浮点数示例

float F = 15213.0

- $15213_{10} = 11101101101101_2 = 1.1101101101101_2 \times 2^{13}$
- **尾数**

$M = 1.\underline{1101101101101}_2$

$\text{frac} = \underline{1101101101101}0000000000_2$

- **阶码**

$E = 13$

$Bias = 127 \quad (2^{e-1}-1) \quad e=8$

$Exp = 140 = 10001100_2$

Hex:	4	6	6	D	B	4	0	0
Binary:	0100	0110	0110	1101	1011	0100	0000	0000
140:	100	0110	0					
15213:			1110	1101	1011	01		

非规格化浮点数（Denormalized）

满足条件

exp = 000…0

其它域的取值

- E = 1 – Bias
- M = 0.xxx…x
  - xxx…x: bits of frac

Bias = $2^{e-1} - 1$, e为指数域的位数
(注：对规格化数而言E = Exp – Bias)

具体示例

- **exp = 000…0, frac = 000…0**
  - 表示0
  - 注意有 +0 与 -0
- **exp = 000…0, frac ≠ 000…0**
  - 表示“非常接近”于0的浮点数
  - 会逐步丧失精度，称为“Gradual underflow（逐步下溢）”
    - 整数："正确"但特定输入下崩溃（溢出）

一些特殊值

满足条件

exp = 111…1

具体示例

exp = 111…1, frac = 000…0

- 表示无穷
- 可用于表示数值的溢出
- 有正无穷与负无穷
- E.g., 1.0/0.0 = +∞, -1.0/0.0 =-∞

exp = 111…1, frac ≠ 000…0

- Not-a-Number (NaN)
- E.g., sqrt(–1), ∞ – ∞, ∞ * 0



目录

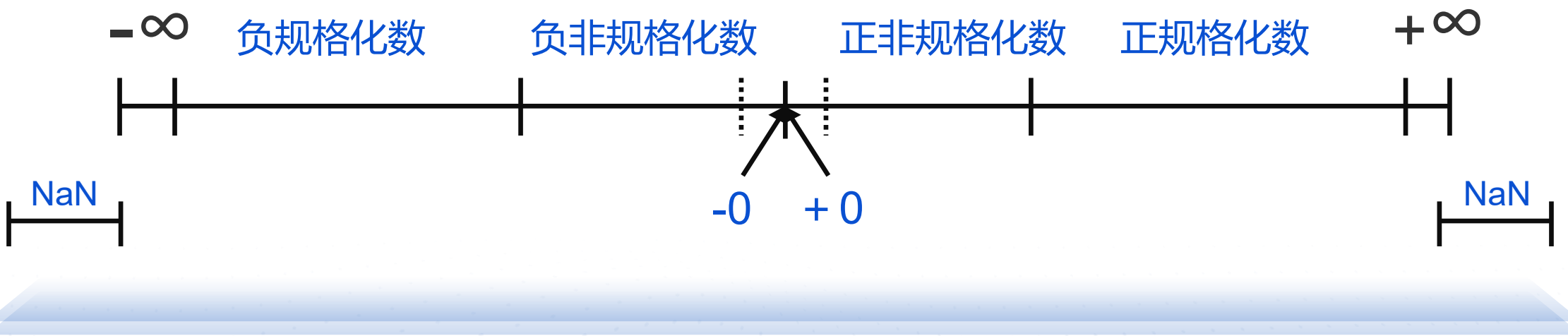
CONTENTS

01 IEEE的浮点数标准

02 Rounding（舍入）

03 C语言中的浮点数

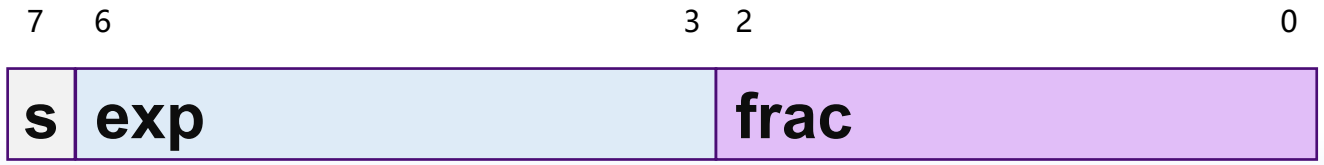
各种浮点数类型在数轴上的相对位置



一种“小”浮点数实例（E4M3）

8位浮点数表示

- 指数域宽度为4比特，小数域宽度为3比特



其他规则符合IEEE 754规范

- 规格化 / 非规格化
- 表示0, NaN与无穷

偏置量是多少？ 7 $(2^{4-1}-1)$

*FP8 格式是一种新的 8 位浮点数格式，主要用于加速深度学习训练和推理，包含两种编码：E4M3 和 E5M2 （2022-2023年提出）
这儿的M表示Mantissa

AI训练/推断中的浮点

格式	指数	小数	总位	使用场景
FP32	8	23	32	训练默认，精度高
FP16	5	10	16	推理加速，范围小
BF16	8	7	16	训练替代FP32，范围同FP32
FP8 (E4M3)	4	3	8	推理，精度优先
FP8 (E5M2)	5	2	8	训练，范围优先

一种“小”浮点数实例

Exp	exp	E	2^E	
0	***0000***	***-6***	***1/64***	***(非规格化数)***
1	0001	*-6*	*1/64*	
2	0010	-5	1/32	
3	0011	*-4*	1/16	
4	0100	-3	1/8	
5	0101	-2	1/4	
6	0110	-1	1/2	
7	0111	0	1	
8	1000	+1	2	
9	1001	+2	4	

Exp	exp	E	2^E
10	1010	+3	8
11	1011	+4	16
12	1100	+5	32
13	1101	+6	64
14	1110	+7	128
15	***1111***	***n/a***	***(inf, NaN)***

取值范围

非规格化数

规格化数

s	exp	frac	E	Value	
0	0000	000	-6	0	
0	0000	001	-6	1/8*1/64 = 1/512	← 最接近于0
0	0000	010	-6	2/8*1/64 = 2/512	
…					
0	0000	110	-6	6/8*1/64 = 6/512	
0	0000	111	-6	7/8*1/64 = 7/512	← 最大的非规格化数
0	0001	000	-6	8/8*1/64 = 8/512	← 最小的规格化数
0	0001	001	-6	9/8*1/64 = 9/512	
…					
0	0110	110	-1	14/8*1/2 = 14/16	
0	0110	111	-1	15/8*1/2 = 15/16	← 最接近于1（小于）
0	0111	000	0	8/8*1 = 1	
0	0111	001	0	9/8*1 = 9/8	← 最接近于1（大于）
0	0111	010	0	10/8*1 = 10/8	
…					
0	1110	110	7	14/8*128 = 224	
0	1110	111	7	15/8*128 = 240	← 最大的规格化数
0	1111	000	n/a	inf	

数轴上的分布



非规格化数　规格化数　无穷

一些特例

Description	exp	frac	Numeric Value
Zero	00…00	00…00	0.0
Smallest Pos. Denorm. Single ≈ 1.4×10^{-45} Double ≈ 4.9×10^{-324}	00…00	00…01	$2^{-\{23,52\}} \times 2^{-\{126,1022\}}$
Largest Denormalized Single ≈ 1.18×10^{-38} Double ≈ 2.2×10^{-308}	00…00	11…11	$(1.0 - \varepsilon) \times 2^{-\{126,1022\}}$
Smallest Pos. Normalized Just larger than largest denormalized	00…01	00…00	$1.0 \times 2^{-\{126,1022\}}$
One	01…11	00…00	1.0
Largest Normalized Single ≈ 3.4×10^{38} Double ≈ 1.8×10^{308}	11…10	11…11	$(2.0 - \varepsilon) \times 2^{\{127,1023\}}$

浮点数的一些编码特性

（几乎）可以直接使用无符号整数的比较方式

反例：

- 必须先比较符号位
- 考虑+0、-0的特例
- 还有NaN的问题…
  - (不考虑符号位的话)，NaN比其他值都大
  - 实际的比较结果如何?

其他情况都可以直接使用无符号整数的比较方式

- 规格化 vs. 非规格化
- 规格化 vs. 无穷



浮点数的一些编码特性

	s	exp	frac	E	Value	
非规格化数	0	0000	000	-6	0	
	0	0000	001	-6	1/8*1/64 = 1/512	← 最接近于0
	0	0000	010	-6	2/8*1/64 = 2/512	
	…					
	0	0000	110	-6	6/8*1/64 = 6/512	
	0	0000	111	-6	7/8*1/64 = 7/512	← 最大的非规格化数
规格化数	0	0001	000	-6	8/8*1/64 = 8/512	← 最小的规格化数
	0	0001	001	-6	9/8*1/64 = 9/512	
	…					
	0	0110	110	-1	14/8*1/2 = 14/16	
	0	0110	111	-1	15/8*1/2 = 15/16	← 最接近于1（小于）
	0	0111	000	0	8/8*1 = 1	
	0	0111	001	0	9/8*1 = 9/8	← 最接近于1（大于）
	0	0111	010	0	10/8*1 = 10/8	
	…					
	0	1110	110	7	14/8*128 = 224	
	0	1110	111	7	15/8*128 = 240	← 最大的规格化数
	0	1111	000	n/a	inf	

给定一个实数，如何给出其浮点数表示?

基本流程

- 首先计算出精确值
- 然后将其转换为所需的精度
  - 可能会溢出（如果指数绝对值很大）
  - 可能需要完成舍入(rounding)操作

各种舍入模式

	$1.40	$1.60	$1.50	$2.50	–$1.50
◦ 向0舍入 (Zero)	$1	$1	$1	$2	–$1
◦ 向下舍入 (Round down)	$1	$1	$1	$2	–$2
◦ 向上舍入 (Round up)	$2	$2	$2	$3	–$1
◦ 向偶数舍入 (Nearest Even)	$1	$2	$2	$2	–$2

向偶数舍入(Round-To-Even)

计算机内默认舍入方式

- 也称为“向最接近值的舍入”，其它方式会产生统计误差（statistically biased）

确定两个可能结果的中间数值的舍入

- 关键的设计决策是确定两个可能结果的中间数值的舍入
  - 确保舍入后的最低有效数字是偶数
  - 举例：保留百分位

1.2349999	1.23	(不到一半)
1.2350001	1.24	(大于一半)
1.2350000	1.24	(恰好一半：为确保最低有效位为偶数，向上舍入)
1.2450000	1.24	(恰好一半：为确保最低有效位为偶数，向下舍入)

向偶数舍入(Round-To-Even)

对于二进制数而言

- “Even”意味着最低有效数字需为0
- 而最低有效数字右侧的位串为100…

实例

舍入到小数点后2位

数值	二进制表示	舍入后	具体的舍入操作	最终结果
2 3/32	10.00011_2	10.00_2	(<1/2—down)	2
2 3/16	10.00110_2	10.01_2	(>1/2—up)	2 1/4
2 7/8	10.11100_2	11.00_2	(1/2—up)	3
2 5/8	10.10100_2	10.10_2	(1/2—down)	2 1/2

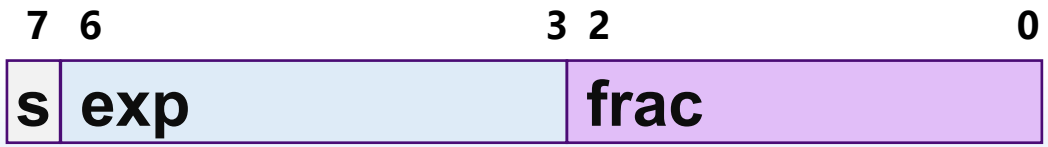
表示$2\frac{5}{8}$

其他类似

步 骤

》具体步骤

- 将数值规格化（前导1）
- 向偶数舍入（round to even）以便符合尾数的位数需求
- 后调整

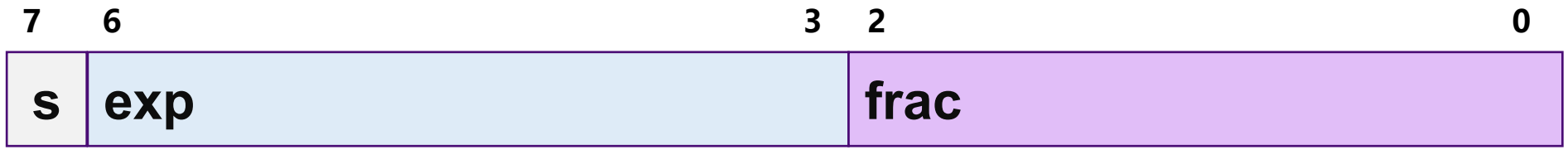


实例

将8位无符号数转换为8位浮点数（指数域宽度为4比特，小数域宽度为3比特）

128	10000000	19	00010011
15	00001111	138	10001010
17	00010001	63	00111111

规格化



数值	二进制表示	有效数	指数
128	10000000	1.0000000	7
15	00001111	1.1110000	3
17	00010001	1.0001000	4
19	00010011	1.0011000	4
138	10001010	1.0001010	7
63	00111111	1.1111100	5

舍 入

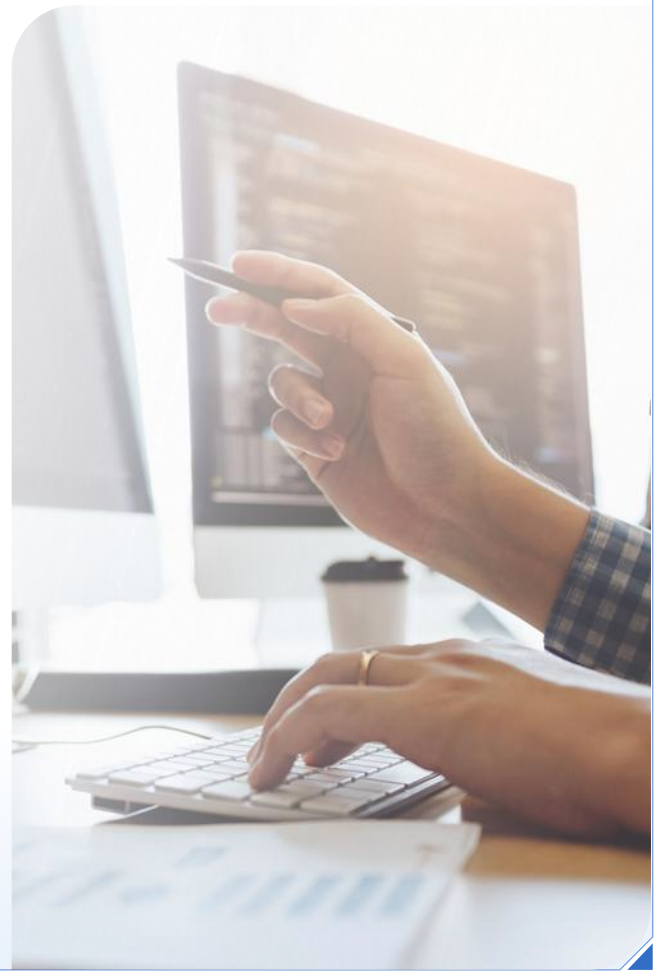
数值	有效数	是否进位?	舍入后的
128	1.0000000	N	1.000
15	1.1110000	N	1.111
17	1.0001000	N	1.000
19	1.0011000	Y	1.010
138	1.0001010	Y	1.001
63	1.1111100	Y	10.000



调整（Postnormalize）

舍入操作可能引起溢出

原始数值	舍入后的	指数	调整	最终结果
128	1.000	7		128
15	1.111	3		15
17	1.000	4		16
19	1.010	4		20
138	1.001	7		144
63	10.000	5	1.000/6	64



C语言中的浮点数

float 单精度浮点数　　double 双精度浮点数

类型转换

当int（32位宽），float, 与double等类型间进行转换时，基本的原则如下:

- double 或 float 转换为 int
  - 尾数部分截断
  - 如果溢出或者浮点数是NaN，则转换结果没有定义
    - 通常置为 Tmin or Tmax
- int转换为double
  - 能够精确转换
- int转换为float
  - 不会溢出，但是可能被舍入



Floating Point Puzzles

以下判断是否成立，如不成立请给出反例

```
int x = ...;
float f = ...;
double d = ...;
```

假设d 与 f 都不是 NaN

- x == (int)(float) x
- x == (int)(double) x
- f == (float)(double) f
- d == (float) d
- f == -(-f);
- 2/3 == 2/3.0
- d < 0.0 → ((d*2) < 0.0)
- d > f → -f > -d
- d * d >= 0.0
- (d+f)-d == f

```c
#include <stdio.h>

int main() {
    float arr[] = {1e10f, 1.0f, -1e10f};
    int n = sizeof(arr) / sizeof(arr[0]);

    // 顺序求和：1.0 被吞没
    float sum1 = 0.0f;
    for (int i = 0; i < n; i++) {
        sum1 += arr[i];
    }
    printf("顺序求和: %.15f\n", sum1);

    // 调整顺序：先加 1.0 和 -1e10f（仍然被吞没）
    float sum2 = 1.0f + (-1e10f) + 1e10f;
    printf("1.0 + (-1e10) + 1e10: %.15f\n", sum2);

    // 正确顺序：
    float sum3 =  (-1e10f) + 1e10f + 1.0f;
    printf("%.15f\n", sum3);  // 1.000000000000000
    return 0;
}
```

33

$$1e10f = (-1_2)^{0_2} \times 10_2^{(10100000_2 - 01111111_2)} \times 1.00101010000001011111001_2$$

$$1.0f = (-1_2)^{0_2} \times 10_2^{(01111111_2 - 01111111_2)} \times 1.00000000000000000000000_2$$